

Implementación Técnica

Normalización de Cargo y Departamento en módulo Empleados

Elaborado por: Emmanuel Arroyo (Tech Lead) Fecha: 21 de abril de 2026 Rama: `enmanuel` Commit: `9eae7e7`

RESUMEN EJECUTIVO

Se implementó la **normalización relacional** de los campos `cargo` y `departamento` en el módulo de Empleados, eliminando el almacenamiento como texto libre y creando **catálogos maestros** con integridad referencial.

Se construyeron módulos CRUD completos accesibles desde la vista de Empleados siguiendo el patrón `TipoProducto` ya establecido en el proyecto, con historial, restauración y validación de dependencias.

Se resolvió además un **bug crítico de Bootstrap 5 con modales anidados** que causaba el cierre del modal padre al cerrar un modal hijo. El fix se aplicó a nivel global del layout, beneficiando automáticamente a todo el proyecto.

15	7	+1900	-377
ARCHIVOS TOCADOS	ARCHIVOS NUEVOS	LÍNEAS AÑADIDAS	LÍNEAS ELIMINADAS

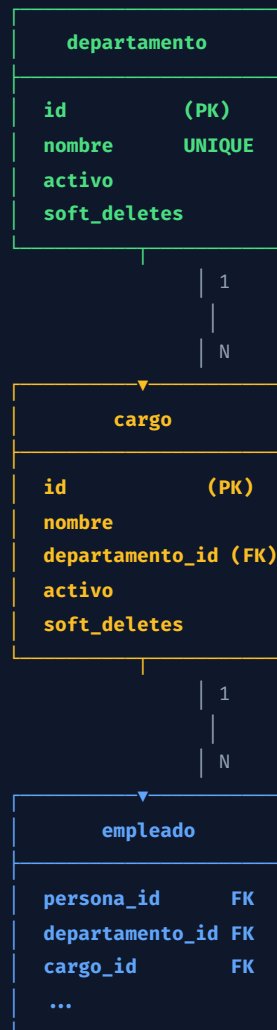
1 Contexto del Problema

En el módulo de **Empleados** de SGPMRJA, los campos `cargo` y `departamento` se almacenaban como texto libre (`varchar(100)`) directamente en la tabla `empleado` , sin catálogos maestros propios ni relación entre ellos.

PROBLEMAS IDENTIFICADOS

- **Inconsistencias de datos:** "Producción" vs "Produccion" vs "PRODUCCION" — cada variante se guardaba como valor único.
- **Sin vínculo lógico:** un empleado podía tener "Costurera" en "Administración", lo cual viola reglas de negocio.
- **Renombrado imposible:** cambiar el nombre de un departamento requería actualizar cada empleado uno por uno.
- **Sin catálogo de valores:** imposible enumerar cargos y departamentos disponibles para auditoría o reportería.
- **Falta de historial:** no había forma de inhabilitar valores obsoletos preservando datos históricos.

2 Modelo Relacional Implementado



Un **departamento** tiene muchos **cargos**. Un cargo pertenece a un único departamento. Esto enforza la regla de negocio: un cargo siempre está contextualizado por su área operativa.

3 Migración Automática de Datos Existentes

La migración `2026_04_21_000003_normalize_departamento_cargo_in_empleado.php` ejecuta los siguientes pasos de forma transaccional y sin intervención manual:

#	PASO	DESCRIPCIÓN
1	Agregar columnas FK nullable	Añade <code>departamento_id</code> y <code>cargo_id</code> como nullable temporalmente.
2	Extraer departamentos únicos	Obtiene valores distintos de <code>empleado.departamento</code> y los inserta en la nueva tabla.
3	Extraer cargos únicos	Obtiene combinaciones distintas <code>(cargo, departamento)</code> y las vincula a su departamento correspondiente.
4	Fallback "Sin clasificar"	Para cargos huérfanos (sin departamento asignado) se crea un departamento de fallback.
5	Actualizar empleados	Rellena <code>departamento_id</code> y <code>cargo_id</code> en cada registro según sus valores de texto previos.
6	Aplicar FKs	Agrega constraints <code>restrictOnDelete</code> para garantizar integridad referencial.
7	Eliminar columnas antiguas	Borra las columnas <code>cargo</code> y <code>departamento</code> (varchar) de <code>empleado</code> .

RESULTADO EN PRODUCCIÓN

2 departamentos (Administración, Producción) y **5 cargos** migrados correctamente. Todos los empleados existentes quedaron reasignados sin pérdida de información y con integridad referencial garantizada.

4 Interfaz: Selects en Cascada

El formulario de Agregar/Editar Empleado ahora implementa la lógica de selección jerárquica que refleja la relación de la base de datos:

Departamento (select primario)

- Muestra todos los departamentos activos del sistema.
- Cuenta con un botón `+` lateral que abre un modal para crear un nuevo departamento on-the-fly.
- Al cambiar su valor, dispara la carga dinámica de cargos mediante AJAX.

Cargo (select dependiente)

- Inicialmente **deshabilitado** mostrando el texto "Elija un departamento".
- Al seleccionar un departamento, se habilita y carga mediante AJAX solo los cargos vinculados a ese departamento.
- El botón `+` para agregar cargo **solo se habilita** cuando hay un departamento seleccionado.
- Al crear un cargo nuevo, queda automáticamente vinculado al departamento activo.

REGLA DE NEGOCIO APLICADA

Una costurera solo puede existir en el departamento de **Producción**. Un contador solo puede existir en **Administración**. El sistema ahora hace imposible asignar combinaciones inválidas a nivel de datos y de interfaz.

5

Módulos CRUD de Gestión (Departamentos y Cargos)

Siguiendo el patrón del módulo `TipoProducto` ya establecido en el proyecto, se agregaron dos botones en el header de la vista de Empleados:

BOTÓN	FUNCIONALIDAD
Departamentos	Abre un modal con DataTable mostrando: Nombre / Cantidad de Cargos / Cantidad de Empleados / Acciones. Incluye toggle Activos/Historial y botón "Nuevo Departamento".
Cargos	Abre un modal con DataTable mostrando: Nombre / Departamento / Cantidad de Empleados / Acciones. Incluye toggle Activos/Historial, filtro por departamento y botón "Nuevo Cargo".

Operaciones disponibles por entidad

OPERACIÓN	DETALLE
Listar	DataTable paginada con filtro por estado (activos / historial).
Crear	Modal con formulario. Valida nombre único (global para depto, por departamento para cargo).
Editar	Modal con formulario pre-poblado. Permite renombrar y, en cargos, reasignar el departamento.
Inhabilitar	SoftDelete con confirmación SweetAlert2. Bloqueado si tiene registros asociados.
Restaurar	Disponible desde la vista "Historial". Recupera el registro con todos sus datos originales.

VALIDACIONES DE INTEGRIDAD

- No se puede inhabilitar un **departamento** que tenga cargos o empleados asociados.
- No se puede inhabilitar un **cargo** que tenga empleados asociados.
- El sistema devuelve mensajes claros explicando el motivo del bloqueo.

Sincronización en vivo

Cualquier cambio realizado en los gestores de Departamentos o Cargos (crear / editar / inhabilitar / restaurar) refresca automáticamente el select del modal de Empleado que está abierto detrás, sin necesidad de recargar la página ni perder el contexto del empleado que el usuario está editando.

6 Fix Crítico: Modales Anidados en Bootstrap 5

BUG DETECTADO

Al abrir un modal pequeño (ej: "Nuevo Departamento") desde dentro de otro modal (ej: "Agregar Empleado") y cerrar el hijo con X o Cancelar, **también se cerraba el modal padre**, haciendo perder todos los datos cargados en el formulario de empleado.

Causa: Bootstrap 5 no soporta modales apilados nativamente (según su propia documentación oficial). Al cerrar cualquier modal, Bootstrap remueve `modal-open` del body de forma incondicional, afectando al padre.

SOLUCIÓN APLICADA

Se adoptó el patrón `modal-hidden-temp` ya existente en el módulo **Inventario** pero se aplicó de forma **global** en `resources/views/admin/layouts/app.blade.php`, cubriendo automáticamente todos los módulos del proyecto.

Código del patch global

```
// Al abrir un modal sobre otro, ocultar el padre temporalmente
$(document).on('show.bs.modal', '.modal', function () {
    var $abiertos = $('.modal.show').not(this);
    if ($abiertos.length > 0) {
        $abiertos.addClass('modal-hidden-temp');
        $(this).data('parentModals', $abiertos);
    }
});

// Al cerrar el hijo, restaurar el padre y el scroll-lock
$(document).on('hidden.bs.modal', '.modal', function () {
    var $padres = $(this).data('parentModals');
    if ($padres && $padres.length > 0) {
        $padres.removeClass('modal-hidden-temp');
    }
    if ($('.modal.show').length > 0) {
        $('body').addClass('modal-open');
    }
});
```

Por qué funciona

- La clase `modal-hidden-temp` aplica `opacity: 0` y `pointer-events: none` al padre.
- La instancia del modal padre **sigue viva** en Bootstrap — nunca se llama a su método `hide()`.
- El usuario solo ve el modal hijo mientras interactúa con él.
- Al cerrar el hijo, se quita la clase y el padre reaparece intacto con todos sus datos.

COBERTURA AUTOMÁTICA

Al estar ubicado en el layout global, este fix protege automáticamente **todos los módulos del proyecto** presentes y futuros que usen modales anidados: Empleados (`showModal` → `addDepartamento/addCargo`), Productos (`showModal` → `addTipo`), Inventario (`createModal` → `modalAddInsumo`), etc.

7 Archivos Modificados / Creados

CATEGORÍA	ARCHIVO	ESTADO
Base de Datos	database/migrations/2026_04_21_000001_create_departamento_table.php	NUEVO
	database/migrations/2026_04_21_000002_create_cargo_table.php	NUEVO
	database/migrations/2026_04_21_000003_normalize_departamento_cargo_in_empleado.php	NUEVO
Modelos	app/Models/Departamento.php	NUEVO
	app/Models/Cargo.php	NUEVO
	app/Models/Empleado.php	MODIFICADO
Controladores	app/Http/Controllers/DepartamentoController.php	NUEVO
	app/Http/Controllers/CargoController.php	NUEVO
	app/Http/Controllers/EmpleadoController.php	MODIFICADO
Servicios	app/Services/EmpleadoService.php	MODIFICADO
Rutas	routes/web.php (+14 rutas nuevas)	MODIFICADO
Vistas	resources/views/admin/empleados/create.blade.php	MODIFICADO
	resources/views/admin/empleados/index.blade.php	MODIFICADO
	resources/views/admin/empleados/reporte_pdf.blade.php	MODIFICADO
	resources/views/admin/layouts/app.blade.php	FIX GLOBAL

8 Comparativa Antes / Después

ANTES

- Texto libre sin validación.
- Imposible renombrar globalmente.
- Cualquier cargo con cualquier departamento.
- Sin historial de valores.
- Solo se podían agregar valores desde un botón + .
- Bug: modales anidados cerraban el padre.
- Inconsistencias "Produccion" / "Producción" / "PRODUCCION".

DESPUÉS

- Catálogos con integridad referencial.
- Un solo registro → un solo cambio global.
- Cargos vinculados al departamento correcto.
- Historial completo + restauración via SoftDeletes.
- CRUD total (crear / editar / inhabilitar / restaurar).
- Fix global de modales anidados para todo el proyecto.
- Nombre único por tabla, case-insensitive.

9 Deuda Técnica Generada

NINGUNA

La implementación siguió los patrones ya establecidos en el proyecto:

- Patrón `TipoProducto` para los módulos CRUD maestros.
- Patrón `modal-hidden-temp` (Inventario) para los modales anidados.
- `SoftDeletes` consistente con los catálogos existentes (ya auditado en migración `ba02`).
- Migración de datos automática y con método `down()` reversible.
- Validaciones server-side + client-side siguiendo el estándar del proyecto.

10 Cierre

La implementación fue consolidada en un único commit coherente con mensaje estructurado, pusheado a la rama `enmanuel` y lista para su revisión y merge a `main` por parte del equipo.

COMMIT DE REFERENCIA

9eae7e7 – feat(empleados): normaliza cargo/departamento + CRUD maestros + fix modales anidados